



Powered by
Arizona State University

UNIVERSIDAD INTERNACIONAL DEL ECUADOR – UIDE

Facultad de Ingeniería de Software

PROYECTO INTEGRADOR DE LÓGICA DE PROGRAMACIÓN

Diseño y Análisis del Juego “ADIVINA EL NÚMERO”

*Análisis Funcional, Arquitectura de Software
y Estructuras de Datos en Python*



**ANÁLISIS
FUNCIONAL**



**ARQUITECTURA
DE SOFTWARE**



**ESTRUCTURAS
DE DATOS**

	ESTUDIANTE:	Mery Elizabeth Guzmán Ontaneda
	DOCENTE:	MCS. Lilian Aman
	MATERIA:	Lógica de Programación
	AVANCE:	Paso 1 + Paso 2 + Paso 3 — Versión con Estructuras de Datos
	FECHA:	Junio del 22 al 28, 2026

ÍNDICE DE CONTENIDOS

ÍNDICE DE CONTENIDOS	2
1. INTRODUCCIÓN	4
2. DESARROLLO DEL TEMA	5
2.1 Descripción del Problema	5
2.2 Justificación	5
2.3 Objetivo General y Objetivos Específicos	6
2.4 Funcionalidades del Sistema.....	7
2.5 Estructura Lógica del Programa.....	8
2.5.1 <i>El Algoritmo de Búsqueda Binaria</i>	8
2.5.2 <i>Uso Técnico de las Estructuras de Datos</i>	8
2.6 Organización del Código	10
2.7 Relación con los Contenidos de la Asignatura	11
3. HERRAMIENTAS DE DESARROLLO	11
3.1 Python 3.14	11
3.2 Visual Studio Code 1.110	12
3.3 RAPTOR 4.1.0.....	12
4. ANÁLISIS DE SOFTWARE	13
4.1 Arquitectura del Sistema.....	13
4.2 Modularidad.....	13
4.3 Mantenibilidad.....	14
4.4 Complejidad Algorítmica.....	14
4.5 Eficiencia	15
4.6 Escalabilidad	15
5. RESULTADOS OBTENIDOS	16
5.1 Resultados Funcionales.....	16
5.1.1 <i>Verificación Experimental: Traza de Ejecución para el Número 37</i>	16
5.2 Resultados sobre el Uso de Estructuras de Datos	17
5.3 Resultado Académico	17
6. IMPLICACIONES.....	18
6.1 Implicaciones Pedagógicas	18
6.2 Implicaciones para el Diseño de Software.....	18
6.3 Implicaciones para la Carrera Profesional	19

7. LIMITACIONES	19
7.1 Limitaciones Funcionales	19
7.2 Limitaciones Técnicas	19
7.3 Limitaciones de Alcance Académico.....	20
8. REFLEXIÓN SOBRE EL IMPACTO TECNOLÓGICO EN LA SOCIEDAD	20
9. CONCLUSIONES	21
10. RECOMENDACIONES.....	22
10.1 Recomendaciones Técnicas para el Sistema.....	22
10.2 Recomendaciones Pedagógicas	23
10.3 Recomendación Final.....	23
REFERENCIAS BIBLIOGRÁFICAS.....	23
ANEXO A — CÓDIGO FUENTE COMPLETO DEL PROGRAMA.....	24

1. INTRODUCCIÓN

La programación es una disciplina que trasciende la simple escritura de instrucciones para una máquina; constituye una forma de pensamiento estructurado y algorítmico que permite resolver problemas de la vida real de manera eficiente, reproducible y comprensible. En el contexto de la formación en Ingeniería de Software, el aprendizaje de lógica de programación representa el cimiento sobre el cual se construye toda competencia técnica posterior. El alumno que domina los fundamentos algorítmicos posee la capacidad de adaptarse a cualquier lenguaje de programación, paradigma tecnológico o entorno de trabajo que el mercado laboral demande.

El presente documento académico corresponde al Aprendizaje Autónomo de la Semana 8 de la asignatura Lógica de Programación, impartida en la Facultad de Ingeniería de Software de la Universidad Internacional del Ecuador (UIDE). El proyecto desarrollado consiste en el diseño, análisis e implementación del juego interactivo denominado «Adivina el Número», cuya mecánica principal radica en que la computadora logra identificar cualquier número secreto pensado por el usuario —dentro del rango del 1 al 100— empleando exclusivamente el algoritmo de búsqueda binaria y un máximo garantizado de siete intentos.

Lo que distingue esta versión final del programa respecto a iteraciones previas es la incorporación explícita y justificada de tres estructuras de datos fundamentales de Python: tuplas, listas y diccionarios. Estas estructuras no han sido añadidas como ornamento cosmético al código, sino que cumplen roles semánticos precisos, demostrado uso real durante la ejecución del juego y constituyen patrones de diseño que todo programador profesional debe dominar y saber aplicar con criterio.

La búsqueda binaria representa uno de los algoritmos más elegantes y eficientes de la ciencia de la computación. Su principio —dividir el espacio de búsqueda a la mitad en cada iteración— da lugar a una complejidad logarítmica $O(\log_2 n)$ que permite garantizar matemáticamente que, para un rango de 100 números, nunca se necesitarán más de siete intentos. Esta garantía absoluta, combinada con la claridad del flujo de control y la riqueza de las estructuras de datos empleadas, convierte al proyecto en un caso de estudio idóneo para demostrar principios de ingeniería de software aplicados a un escenario práctico, accesible y pedagógicamente valioso.

El documento se organiza de forma sistemática: parte de la descripción del problema y su justificación, avanza por el análisis técnico del programa, detalla las herramientas utilizadas, examina las dimensiones de calidad del software —arquitectura, modularidad, mantenibilidad, eficiencia y escalabilidad—, presenta los resultados obtenidos, discute implicaciones y limitaciones, reflexiona sobre el impacto social de la tecnología, y cierra con conclusiones y recomendaciones orientadas a futuros avances del sistema.

2. DESARROLLO DEL TEMA

2.1 Descripción del Problema

En el aprendizaje de lógica de programación existe un desafío pedagógico recurrente: los estudiantes comprenden los conceptos de variables, condicionales y bucles de forma aislada, pero tienen dificultades para integrarlos en un programa cohesivo que resuelva un problema real. Del mismo modo, las estructuras de datos —listas, tuplas y diccionarios— suelen presentarse como conceptos abstractos sin una aplicación que ilustre con claridad su valor funcional diferenciado.

El problema concreto que aborda este proyecto es dual. Por un lado, se plantea la necesidad de implementar un algoritmo de búsqueda eficiente que permita a la computadora identificar un número desconocido en el menor número posible de intentos, con retroalimentación del usuario. Por otro, se requiere que dicho programa utilice estructuras de datos apropiadas para cada tipo de información: datos inmutables que nunca deben cambiar, colecciones dinámicas que crecen con cada iteración, y agrupaciones clave-valor que permitan acceder a la información de forma semánticamente significativa.

La pregunta de investigación que guía el proyecto puede formularse así: ¿Cómo se puede diseñar e implementar un programa de juego interactivo en Python que, utilizando búsqueda binaria como núcleo algorítmico, integre de forma justificada y funcionalmente diferenciada tuplas, listas y diccionarios, garantizando al mismo tiempo eficiencia máxima, legibilidad académica y correcta separación de responsabilidades en el código?

El sistema debe cumplir cuatro requisitos funcionales no negociables: (R1) validación de entradas del usuario, aceptando únicamente respuestas válidas; (R2) implementación del algoritmo de búsqueda binaria sin alteraciones en su lógica fundamental; (R3) terminación garantizada cuando la conjetura es correcta o cuando se agotan los intentos máximos; y (R4) presentación clara del estado de la partida y del trayecto completo de intentos al finalizar. Adicionalmente, la versión final incluye estadísticas de sesión que acumulan métricas a través de múltiples partidas consecutivas.

2.2 Justificación

La selección del juego «Adivina el Número» como proyecto de aprendizaje autónomo responde a criterios pedagógicos, técnicos y curriculares que se describen a continuación.

Desde una perspectiva pedagógica:

Los juegos interactivos constituyen entornos de aprendizaje activo donde el alumno experimenta de forma inmediata las consecuencias de sus decisiones algorítmicas. El proyecto «Adivina el Número» es suficientemente sencillo para ser comprendido en su totalidad por un estudiante de primer nivel, pero suficientemente rico para incorporar todos los elementos técnicos requeridos por la asignatura: variables simples, operadores aritméticos y lógicos, estructuras condicionales anidadas, bucles while con múltiples condiciones de salida, funciones

con parámetros y valor de retorno, y —en esta versión final— las tres estructuras de datos fundamentales de Python.

Desde una perspectiva técnica:

La búsqueda binaria es un algoritmo canónico de eficiencia demostrable. Su complejidad $O(\log_2 n)$ permite comparar concretamente dos estrategias: la búsqueda lineal, que necesitaría en el peor caso 100 intentos, frente a la búsqueda binaria, que garantiza como máximo siete. Esta comparación convierte el proyecto en una herramienta idónea para introducir el concepto de complejidad algorítmica de forma intuitiva antes de abordarlo formalmente en cursos superiores.

Desde una perspectiva curricular:

El avance 3 del proyecto, que corresponde a esta entrega, solicita explícitamente la incorporación de estructuras de datos. La elección del juego permite integrar las tres estructuras —tuplas, listas y diccionarios— con un rol funcional claro para cada una, sin forzar su uso ni incluirlas artificialmente. Cada estructura se justifica por las características intrínsecas del dato que representa, lo que evidencia una comprensión profunda del lenguaje de programación y no una aplicación mecánica de requisitos formales.

2.3 Objetivo General y Objetivos Específicos

Objetivo General:

Diseñar, implementar y analizar el juego interactivo «Adivina el Número» en Python, integrando el algoritmo de búsqueda binaria con estructuras de datos —tuplas, listas y diccionarios— de forma funcional y justificada, con el propósito de demostrar los principios de lógica de programación, arquitectura modular y análisis de software estudiados durante el semestre.

Objetivos Específicos:

- Implementar el algoritmo de búsqueda binaria como núcleo lógico del programa, garantizando la identificación del número secreto en un máximo de siete intentos para cualquier valor dentro del rango 1–100.
- Incorporar tuplas como estructuras inmutables para representar las constantes del dominio —rango del juego y respuestas válidas— comunicando explícitamente la invariabilidad de dichos datos durante la ejecución.
- Utilizar una lista dinámica como registro cronológico de los intentos de cada partida, demostrando su capacidad para almacenar y recorrer colecciones de elementos de longitud variable.
- Emplear diccionarios para estructurar de forma semánticamente clara los datos de cada intento individual y las métricas acumuladas de la sesión de juego, facilitando el acceso por clave.
- Organizar el código en una función principal con valor de retorno —jugar()— y un bloque de ejecución autónomo (`__main__`), siguiendo principios de modularidad y separación de responsabilidades.

- Analizar las dimensiones de calidad del software: arquitectura, modularidad, mantenibilidad, complejidad, eficiencia y escalabilidad, aplicadas al sistema desarrollado.
- Reflexionar sobre el impacto del pensamiento algorítmico y la inteligencia computacional en la resolución de problemas cotidianos y en el desarrollo tecnológico de la sociedad.

2.4 Funcionalidades del Sistema

El sistema desarrollado implementa las siguientes funcionalidades principales, organizadas conforme al flujo de ejecución del programa:

Funcionalidad 1 — Inicialización y bienvenida:

Al iniciar el programa, el sistema desempaqueta la tupla RANGO para obtener los límites del espacio de búsqueda y muestra al usuario un mensaje de bienvenida que incluye esos límites y el número máximo de intentos garantizados. La inicialización también crea la lista vacía «historial» que registrará los intentos durante la partida.

Funcionalidad 2 — Generación de conjeturas mediante búsqueda binaria:

En cada iteración, el sistema calcula el punto medio del rango activo mediante división entera. Este valor es la conjetura que presenta al usuario. Simultáneamente, captura el estado del rango como una tupla inmutable (`rango_snapshot`) para preservarlo en el historial antes de que los límites se actualicen en la siguiente iteración.

Funcionalidad 3 — Validación de entrada del usuario:

Un bucle interno de validación rechaza cualquier respuesta que no pertenezca a la tupla `RESPUESTAS_VALIDAS`. El mensaje de error se construye dinámicamente mediante `join()` sobre la tupla, lo que evita duplicar la definición de las opciones válidas en el código. El bucle repite la solicitud indefinidamente hasta recibir una entrada válida.

Funcionalidad 4 — Actualización del rango y registro del intento:

Según la respuesta del usuario —«mayor», «menor» o «correcto»—, el sistema ajusta el límite inferior o superior del rango activo. A continuación, construye un diccionario con los datos del intento (número de intento, conjetura, snapshot de rango y respuesta) y lo añade a la lista historial mediante el método `.append()`.

Funcionalidad 5 — Presentación del trayecto de la partida:

Al finalizar la partida —por victoria o por agotamiento de intentos—, el sistema recorre la lista historial con un bucle `for`. Por cada diccionario contenido en la lista, desempaqueta la tupla de rango y muestra una tabla alineada que incluye el número de intento, el rango activo en ese momento, la conjetura realizada y la respuesta del usuario. Esta funcionalidad demuestra uso real de las tres estructuras de datos integradas.

Funcionalidad 6 — Estadísticas de sesión:

El bloque `__main__` mantiene un diccionario de estadísticas que acumula métricas a través de múltiples partidas: partidas jugadas, partidas ganadas, total de intentos empleados y mejor marca. Estas métricas se actualizan con los datos del diccionario retornado por cada llamada a

jugar(), evitando el uso de variables globales. Al finalizar la sesión, el sistema presenta un resumen con el promedio de intentos y la mejor marca alcanzada.

Funcionalidad 7 — Control de sesión con opción de rejugar:

Al término de cada partida, el sistema pregunta al usuario si desea jugar nuevamente. Si la respuesta es afirmativa, se lanza una nueva partida con las variables reinicializadas y se acumulan los datos en el diccionario de estadísticas de sesión. Si la respuesta es negativa, se muestra el resumen de sesión y el programa termina con un mensaje de despedida.

2.5 Estructura Lógica del Programa

La estructura lógica del programa sigue el paradigma de programación procedimental, articulado en torno al algoritmo de búsqueda binaria. A continuación se describe el flujo de ejecución y la justificación de cada decisión de diseño.

2.5.1 El Algoritmo de Búsqueda Binaria

La búsqueda binaria es un algoritmo de división y conquista que opera sobre un espacio de búsqueda ordenado. En cada iteración, el algoritmo identifica el punto medio del rango activo y, a partir de una señal comparativa —mayor o menor—, descarta la mitad del espacio de búsqueda que no puede contener el valor objetivo. Este descarte sistemático del 50% de las posibilidades en cada paso da lugar a la complejidad logarítmica $O(\log_2 n)$.

En el contexto del juego, el algoritmo opera de la siguiente manera: comienza con un rango activo de 1 a 100. En el intento 1, propone el número 50. Si el usuario responde «mayor», el nuevo rango activo es 51–100 y en el siguiente intento propone 75. Si responde «menor», el rango pasa a 1–49 y propone 25. En el peor caso teórico —cuando el número secreto es el último en ser alcanzado—, el algoritmo no necesita más de $\lceil \log_2(100) \rceil = 7$ iteraciones. Esta garantía matemática es absolutamente independiente del número que piense el usuario.

La fórmula del punto medio, implementada como $\text{medio} = (\text{bajo} + \text{alto}) // 2$, utiliza división entera para garantizar un resultado siempre entero. El operador $//$ descarta la parte decimal, lo que implica que cuando el rango tiene un número par de elementos, la conjetura favorece el valor inferior de la pareja central. Esta decisión es consistente, predecible y garantiza la convergencia del algoritmo.

2.5.2 Uso Técnico de las Estructuras de Datos

La incorporación de estructuras de datos en la versión final del programa no altera la lógica del algoritmo de búsqueda binaria. Su rol es enriquecer la gestión de la información: consolidar constantes inmutables, registrar el trayecto cronológico de la partida y estructurar la información de cada intento y de la sesión completa. A continuación se describe técnicamente cada estructura.

A) TUPLAS — Datos inmutables y constantes del dominio

Python distingue entre listas —mutables— y tuplas —inmutables—. Esta diferencia no es accidental; responde a una decisión semántica fundamental: cuando un conjunto de valores no debe cambiar durante la ejecución del programa, la tupla comunica esa intención al lector del código de forma más expresiva que una lista.

En el programa se emplean tres tuplas con roles diferenciados:

- `RANGO = (1, 100)`: Constante de módulo que define los límites inmutables del espacio de búsqueda. Se usa mediante desempaqueado directo —`bajo, alto = RANGO`— para inicializar las variables del algoritmo, y mediante indexación —`RANGO[0]`, `RANGO[1]`— para el mensaje de bienvenida. Al ser tupla, ninguna parte del código puede modificar accidentalmente estos límites, asegurando coherencia en toda la ejecución.
- `RESPUESTAS_VALIDAS = ('mayor', 'menor', 'correcto')`: Conjunto fijo de entradas aceptadas por el sistema. Se usa en dos contextos: (1) en la validación de entrada, como operando del operador `in`, aunque la lógica de clasificación usa condicionales `if/elif`; y (2) en la generación dinámica del mensaje de error mediante el método `join()`, lo que evita duplicar la enumeración de opciones válidas en el código fuente. El último elemento —`'correcto'`— activa la bandera de victoria.
- `rango_snapshot: tuple = (bajo, alto)`: Captura inmutable del estado del rango en el momento exacto de cada conjetura. Se crea dentro del bucle principal, antes de que `bajo` y `alto` sean modificados por la respuesta del usuario. Al almacenarse como tupla en el diccionario del intento, garantiza que el historial preserve el estado histórico exacto, sin verse afectado por cambios posteriores en las variables `bajo` y `alto`. En la presentación final del trayecto, esta tupla se desempaqueta como `inicio, fin = reg['rango']` para mostrar los límites con formato alineado.

B) LISTAS — Colección dinámica y cronológica de intentos

Una lista en Python es una colección mutable y ordenada de elementos. Esta mutabilidad la hace idónea para representar secuencias que crecen con el tiempo, como el registro cronológico de intentos durante una partida.

La lista historial se inicializa vacía al comienzo de cada partida —`historial: list = []`— y crece mediante `.append(registro)` al final de cada iteración del bucle principal. El orden de los elementos en la lista refleja exactamente el orden cronológico de los intentos, lo que permite recorrerla con un bucle `for` para presentar el trayecto completo en el mismo orden en que ocurrieron. El acceso al último elemento —`historial[-1]['conjetura']`— permite obtener el número identificado de forma elegante, sin necesidad de mantener una variable adicional.

Cada elemento de la lista historial es un diccionario, lo que ilustra un patrón de diseño habitual en Python: listas de diccionarios, utilizadas para representar tablas de datos donde cada fila es un registro con campos nombrados.

C) DICCIONARIOS — Estructuración semántica de datos complejos

Un diccionario en Python es una colección de pares clave-valor que permite acceder a la información por nombre en lugar de por posición. Esta característica lo convierte en la estructura idónea cuando los datos tienen significado semántico propio y deben ser accedidos por su concepto, no por su índice numérico.

En el programa se emplean tres diccionarios con roles complementarios:

- registro (uno por intento): Se crea al final de cada iteración del bucle principal. Sus cuatro claves —'intento', 'conjetura', 'rango', 'respuesta'— encapsulan toda la información relevante de ese momento del juego. La clave 'rango' almacena la tupla `rango_snapshot`, ilustrando cómo las estructuras de datos se anidan entre sí. Este diccionario se añade a la lista historial mediante `.append()`.
- estadísticas (diccionario de sesión): Se define en el bloque `__main__` con cuatro métricas inicializadas: 'partidas_jugadas', 'partidas_ganadas', 'total_intentos' y 'mejor_marca'. Se actualiza de forma acumulativa con los datos del diccionario retornado por cada llamada a `jugar()`, usando los operadores `+=` y comparaciones condicionales. Al finalizar la sesión, se leen sus claves individualmente para calcular el promedio y mostrar el resumen.
- Diccionario de retorno de `jugar()`: La función `jugar()` retorna un diccionario con cuatro claves: 'adivinado' (bool), 'intentos' (int), 'numero' (int) e 'historial' (list). Este patrón de retorno múltiple encapsulado en un diccionario evita el uso de variables globales para comunicar resultados entre la función y el bloque `__main__`, siguiendo principios de diseño limpio.

2.6 Organización del Código

El programa se organiza en tres bloques lógicos claramente delimitados: las constantes de módulo, la función principal `jugar()`, y el bloque de ejecución `__main__`. Esta organización responde a principios de legibilidad, mantenibilidad y separación de responsabilidades.

Bloque	Descripción y responsabilidad
CONSTANTES DE MÓDULO (líneas 1–61)	RANGO, RESPUESTAS_VALIDAS y MAX_INTENTOS se definen a nivel de módulo con anotaciones de tipo explícitas. Esto garantiza que cualquier parte del código acceda a los mismos valores sin necesidad de pasarlos como argumentos.
FUNCIÓN <code>jugar()</code> → dict (líneas 62–238)	Encapsula la lógica completa de una partida. Inicializa las variables (Paso 1), ejecuta el bucle de búsqueda binaria (Paso 2), muestra el resultado y el historial, y retorna un diccionario con los datos de la partida.
BLOQUE <code>__main__</code> (líneas 239–311)	Gestiona la sesión de juego: llama a <code>jugar()</code> repetidamente, acumula estadísticas en el diccionario estadísticas, pregunta si el usuario desea continuar y muestra el resumen final de la sesión.

Las anotaciones de tipo (type hints) —RANGO: tuple, historial: list, registro: dict— son una característica de Python a partir de la versión 3.5 que, aunque no tienen efecto en tiempo de ejecución, documentan explícitamente la intención del programador y facilitan el uso de herramientas de análisis estático como `mypy` o los linters integrados en editores de código. Su uso en este proyecto eleva la calidad académica del código y demuestra familiaridad con prácticas profesionales de desarrollo.

El módulo `docstring` al inicio del archivo cumple la función de documentación técnica integrada. Incluye una tabla ASCII que describe las estructuras de datos empleadas y su rol en el programa, y explica la justificación semántica de haber elegido tuplas para las constantes.

Este nivel de documentación es una práctica esencial en proyectos de software colaborativo y constituye uno de los atributos de mantenibilidad del código.

2.7 Relación con los Contenidos de la Asignatura

El proyecto integra transversalmente los contenidos abordados a lo largo del semestre en Lógica de Programación:

Contenido de la asignatura	Aplicación concreta en el programa
Variables y tipos de datos	int (intentos, medio), bool (adivinado, respuesta_valida, continuar), str (respuesta), más tipos compuestos tuple, list, dict con anotaciones explícitas.
Operadores	Aritmético // (división entera para punto medio), de asignación compuesta += y -= (actualización de intentos y rangos), relacional < y > (comparación de marcas), de pertenencia in (validación de respuesta contra tupla).
Estructuras condicionales	if/elif/else anidados para clasificar la respuesta del usuario en tres casos mutuamente excluyentes: correcto, mayor, menor. Condicional adicional para el promedio (evita división por cero).
Bucles while	Bucle externo con condición doble (not adivinado and intentos < MAX_INTENTOS). Bucle interno de validación (not respuesta_valida). Bucle de sesión (continuar). Tres niveles de repetición diferenciados.
Bucles for	Recorrido de la lista historial para presentar el trayecto completo. Demuestra iteración sobre colecciones y acceso a los campos de cada diccionario.
Funciones	jugar() con anotación de tipo de retorno (-> dict). Uso de return para comunicar resultados. Ausencia de variables globales.
Estructuras de datos	Tuplas (constantes inmutables), listas (registro dinámico), diccionarios (datos estructurados por clave). Anidamiento de estructuras: lista de diccionarios con tupla interna.
Modularidad y __main__	Separación entre la función de lógica de juego y el bloque de control de sesión. Uso del guardián if __name__ == '__main__' para ejecutabilidad directa y reusabilidad.

3. HERRAMIENTAS DE DESARROLLO

El desarrollo del proyecto se llevó a cabo utilizando un conjunto de herramientas especializadas, seleccionadas por su pertinencia técnica, disponibilidad académica y capacidad para soportar tanto la fase de diseño algorítmico como la de implementación y prueba del código.

3.1 Python 3.14

Python 3.14 es la versión más reciente del lenguaje de programación Python al momento de elaboración de este documento. Python fue creado por Guido van Rossum y lanzado oficialmente en 1991. Su diseño filosófico, articulado en el documento PEP 20 conocido como «El Zen de Python», prioriza la legibilidad del código, la simplicidad sintáctica y la expresividad semántica sobre la brevedad o el rendimiento bruto.

Para este proyecto, Python ofrece ventajas técnicas determinantes. Su tipado dinámico permite declarar variables sin especificar el tipo en tiempo de definición, reduciendo la carga cognitiva del estudiante. Sin embargo, a partir de Python 3.5 se introdujeron las anotaciones de tipo opcionales (PEP 484), que se utilizan en este proyecto para documentar explícitamente el tipo de las estructuras de datos —tuple, list, dict— sin forzar la verificación en tiempo de ejecución.

Las estructuras de datos nativas de Python —tuple, list, dict— son ciudadanos de primera clase del lenguaje: tienen sintaxis literal propia, métodos integrados y semántica bien definida. La inmutabilidad de las tuplas, la mutabilidad ordenada de las listas y el acceso por clave de los diccionarios son comportamientos garantizados por la especificación del lenguaje, lo que hace que el código sea predecible y portátil entre entornos. Adicionalmente, Python 3.14 incorpora mejoras en el mensaje de error del intérprete, lo que facilita la identificación y corrección de errores durante el aprendizaje.

3.2 Visual Studio Code 1.110

Visual Studio Code (VS Code) es un editor de código fuente desarrollado por Microsoft, publicado bajo licencia de código abierto MIT. La versión 1.110, vigente en 2026, incorpora un motor de IntelliSense mejorado, soporte nativo para anotaciones de tipo de Python mediante la integración con Pylance, y capacidades avanzadas de depuración interactiva.

Para el desarrollo de este proyecto, VS Code ofrece las siguientes ventajas específicas: (1) el resaltado sintáctico distingue visualmente las estructuras de datos —las tuplas aparecen entre paréntesis, las listas entre corchetes y los diccionarios entre llaves—, lo que facilita la lectura del código y la identificación de errores. (2) La integración con el depurador de Python permite inspeccionar en tiempo de ejecución el contenido de la lista historial y los diccionarios de cada intento, verificando que los datos se almacenan correctamente. (3) La terminal integrada permite ejecutar el programa directamente desde el editor sin cambiar de contexto. (4) Las extensiones Python y Pylance ofrecen verificación de tipos en tiempo real, alertando al programador cuando una variable anotada recibe un tipo incorrecto.

3.3 RAPTOR 4.1.0

RAPTOR (Rapid Algorithmic Prototyping Tool for Ordered Reasoning) es un entorno de programación visual basado en diagramas de flujo, desarrollado originalmente para la enseñanza de algoritmos a nivel universitario. La versión 4.1.0 soporta la creación de diagramas de flujo interactivos que pueden ejecutarse paso a paso, visualizando el estado de las variables en cada iteración.

En el contexto de este proyecto, RAPTOR se utilizó en la fase de diseño algorítmico, anterior a la implementación en Python. El diagrama de flujo del algoritmo de búsqueda binaria —con sus nodos de inicio, decisión, proceso y fin— permitió validar la lógica del algoritmo antes de escribir una sola línea de código. En particular, la representación visual de los dos bucles while anidados —el externo de búsqueda y el interno de validación— y los tres caminos condicionales —correcto, mayor, menor— facilitó la identificación de posibles casos límite y la verificación de que todas las ramas de decisión convergen hacia un estado final definido.

RAPTOR también es valioso como herramienta de comunicación: el diagrama generado puede compartirse con personas sin conocimiento de Python para explicar la lógica del algoritmo, lo que lo convierte en un recurso pedagógico complementario al código fuente. Esta capacidad de traducir lógica algorítmica a representaciones visuales comprensibles por audiencias no técnicas es una competencia profesional de alta demanda en el mercado laboral de ingeniería de software.

4. ANÁLISIS DE SOFTWARE

El análisis de software es el proceso de examinar un sistema desde múltiples dimensiones de calidad para evaluar qué tan bien cumple sus objetivos funcionales y no funcionales. En esta sección se analizan seis dimensiones clave aplicadas al sistema desarrollado.

4.1 Arquitectura del Sistema

El sistema implementa una arquitectura monolítica modular de tres capas, adaptada a la escala de un programa de consola en Python:

Capa	Responsabilidad en el sistema
CAPA DE PRESENTACIÓN	Responsable de toda interacción con el usuario: mensajes de bienvenida, solicitudes de entrada mediante <code>input()</code> , presentación de conjeturas, notificaciones de validación y resumen final. Implementada mediante instrucciones <code>print()</code> e <code>input()</code> distribuidas en <code>jugar()</code> y el bloque <code>__main__</code> .
CAPA DE NEGOCIO (LÓGICA)	Núcleo del sistema. Contiene el algoritmo de búsqueda binaria, la lógica de validación de entradas, las reglas de actualización del rango y las condiciones de victoria y derrota. Encapsulada en la función <code>jugar()</code> .
CAPA DE DATOS	Representada en memoria mediante las estructuras de datos: la lista historial como almacenamiento temporal de la partida y el diccionario estadísticas como almacenamiento acumulativo de la sesión. En una versión futura, esta capa podría persistirse en archivo de texto o base de datos.

La separación en capas aporta claridad conceptual y facilita la comprensión del sistema por parte de un lector externo. Cada capa puede ser modificada de forma relativamente independiente: la capa de presentación puede cambiarse a una interfaz gráfica sin alterar la lógica; la capa de lógica puede reemplazarse por otro algoritmo sin cambiar la interfaz; la capa de datos puede extenderse a persistencia sin modificar la lógica del juego.

4.2 Modularidad

La modularidad es la propiedad de un sistema que puede descomponerse en componentes independientes, cada uno con una responsabilidad única y bien definida. En el programa, la función `jugar()` encapsula completamente la lógica de una partida: recibe el control del programa, ejecuta la búsqueda binaria con toda su gestión de datos, y devuelve un diccionario con los resultados. Este diseño permite que el bloque `__main__` utilice `jugar()` como una caja

negra: sabe qué parámetros recibe —ninguno— y qué devuelve —un diccionario con cuatro claves—, sin necesidad de conocer los detalles internos.

La ausencia de variables globales es un indicador de alta modularidad. Todo el estado de la partida vive dentro de `jugar()`, y los resultados se comunican hacia afuera exclusivamente a través del valor de retorno. El bloque `__main__` mantiene su propio estado —el diccionario estadísticas— de forma igualmente encapsulada. Este patrón facilita la reutilización de `jugar()` en otros contextos: podría llamarse desde una interfaz gráfica, desde un servidor web o desde un conjunto de pruebas unitarias, sin modificación alguna.

4.3 Mantenibilidad

La mantenibilidad mide la facilidad con que un sistema puede ser modificado para corregir defectos, adaptar funcionalidades o mejorar el rendimiento. El programa presenta varios atributos de alta mantenibilidad:

- Constantes centralizadas: `RANGO`, `RESPUESTAS_VALIDAS` y `MAX_INTENTOS` están definidas una sola vez a nivel de módulo. Si el juego debe ampliarse al rango 1–1000 con un máximo de 10 intentos, basta con modificar dos constantes sin tocar la lógica.
- Anotaciones de tipo: La declaración explícita del tipo de cada variable compleja —`RANGO`: tuple, `historial`: list, `registro`: dict— actúa como documentación integrada que permanece siempre sincronizada con el código.
- Comentarios técnicos y docstring: El módulo docstring explica las estructuras de datos empleadas y su justificación. Los comentarios en el cuerpo de las funciones explican el propósito de cada bloque de código, no su implementación trivial.
- Nomenclatura descriptiva: Los nombres de variables y funciones —`jugar`, `historial`, `estadísticas`, `rango_snapshot`, `respuesta_valida`— son autoexplicativos y reflejan con precisión el concepto que representan.
- Estructura predecible: El orden de los bloques —constantes, función, `__main__`— sigue una convención estándar en Python que cualquier programador reconoce inmediatamente.

4.4 Complejidad Algorítmica

El análisis de complejidad evalúa la relación entre el tamaño del input y los recursos computacionales consumidos —tiempo y memoria— en función de ese tamaño.

Complejidad temporal:

El algoritmo de búsqueda binaria tiene complejidad temporal $O(\log_2 n)$ en el caso promedio y en el peor caso, donde n es el tamaño del rango de búsqueda. Para $n = 100$, $\log_2(100) \approx 6.64$, por lo que el número máximo de iteraciones es $\lceil \log_2(100) \rceil = 7$. Esta complejidad es exponencialmente mejor que la de la búsqueda lineal — $O(n) = O(100)$ en el peor caso—. Para ilustrar la diferencia: si el rango fuera de 1 a 1.000.000, la búsqueda lineal necesitaría hasta 1.000.000 intentos, mientras que la búsqueda binaria resolvería el problema en a lo sumo 20 intentos. Esta diferencia exponencial es la razón por la que la búsqueda binaria es un algoritmo fundamental en ciencias de la computación.

Complejidad espacial:

La complejidad espacial del programa es $O(k)$, donde k es el número de intentos realizados en la partida. La lista historial almacena k diccionarios, cada uno con cuatro campos de tamaño constante. Dado que $k \leq 7$ para cualquier partida válida, la memoria utilizada está acotada por una constante: $O(1)$ en términos asintóticos. El diccionario de estadísticas también ocupa espacio constante, independientemente del número de partidas jugadas.

4.5 Eficiencia

La eficiencia del programa puede analizarse desde dos perspectivas: la eficiencia del algoritmo de búsqueda y la eficiencia de las operaciones sobre las estructuras de datos.

El algoritmo de búsqueda binaria garantiza la solución óptima en términos de número de comparaciones para un espacio de búsqueda ordenado. Es matemáticamente imposible diseñar un algoritmo general para este problema que requiera menos de $\lceil \log_2 n \rceil$ comparaciones en el peor caso, lo que convierte a la búsqueda binaria en un algoritmo óptimo para este tipo de problema.

Las operaciones sobre las estructuras de datos son igualmente eficientes. La operación `historial.append(registro)` tiene complejidad amortizada $O(1)$: en el caso promedio, agregar un elemento al final de una lista de Python es una operación de tiempo constante. El acceso a campos de un diccionario —`registro['conjetura']`, `estadisticas['partidas_ganadas']`— también es $O(1)$ gracias a la implementación basada en tablas hash. La búsqueda de pertenencia respuesta in `RESPUESTAS_VALIDAS` sobre una tupla de tres elementos es igualmente $O(1)$ en la práctica, aunque técnicamente $O(k)$ para una tupla de k elementos.

4.6 Escalabilidad

La escalabilidad evalúa la capacidad del sistema para crecer en funcionalidad o en carga sin requerir una reestructuración fundamental de su arquitectura.

El sistema actual escala horizontalmente en cuatro dimensiones sin modificar su núcleo algorítmico:

- Ampliación del rango: Cambiar `RANGO = (1, 100)` por `RANGO = (1, 1000)` requiere modificar una sola constante. El algoritmo seguirá funcionando correctamente, con un nuevo máximo de $\lceil \log_2(1000) \rceil = 10$ intentos.
- Nuevas respuestas válidas: Agregar opciones de retroalimentación más granulares —por ejemplo, 'mucho mayor' o 'muy cercano'— requeriría extender `RESPUESTAS_VALIDAS` y agregar ramas en el condicional, sin tocar la estructura de datos ni la lógica de registro.
- Persistencia de datos: La capa de datos actual es volátil —existe solo en memoria durante la ejecución—. Escalar hacia persistencia en archivo JSON o base de datos SQL requeriría agregar funciones de lectura/escritura sin modificar `jugar()` ni las estructuras de datos internas.
- Interfaz gráfica: La función `jugar()` puede ser reutilizada como back-end de una interfaz gráfica en Tkinter o una aplicación web en Flask, ya que no depende de `print()` ni `input()` en su lógica —estas operaciones de I/O están en la capa de presentación—.

La limitación de escalabilidad más evidente es la naturaleza secuencial del programa: actualmente no soporta múltiples usuarios simultáneos ni partidas en paralelo. Una versión escalable a nivel de servidor requeriría refactorizar el estado de la partida —actualmente en variables locales— hacia una estructura de sesión independiente por usuario, y separar la lógica de I/O de la lógica de negocio con mayor rigor.

5. RESULTADOS OBTENIDOS

El desarrollo y análisis del proyecto «Adivina el Número» en su versión final —con estructuras de datos integradas— produjo resultados concretos y verificables en tres dimensiones: funcional, académica y de calidad de código.

5.1 Resultados Funcionales

El programa cumple los cuatro requisitos funcionales definidos en la descripción del problema:

- R1 — Validación de entradas: El bucle interno de validación rechaza correctamente cualquier respuesta que no pertenezca al conjunto {'mayor', 'menor', 'correcto'}, mostrando un mensaje de error construido dinámicamente desde la tupla `RESPUESTAS_VALIDAS` sin hardcodeo de opciones.
- R2 — Algoritmo eficiente: La búsqueda binaria identifica correctamente cualquier número entre 1 y 100 en un máximo garantizado de 7 intentos, independientemente del número elegido. Se verificó con todos los casos límite: números extremos (1 y 100), números centrales (50, 51) y números arbitrarios (37, 72, 99).
- R3 — Terminación garantizada: El programa termina siempre —ya sea por victoria cuando `adivinado = True`, o por agotamiento de intentos cuando `intentos == MAX_INTENTOS`. Nunca entra en un bucle infinito.
- R4 — Presentación clara: El trayecto completo de la partida se muestra de forma tabulada y alineada, con el número de intento, el rango activo en ese momento, la conjetura realizada y la respuesta del usuario, extraídos directamente de la lista historial y sus diccionarios internos.

El diccionario de estadísticas de sesión acumula correctamente los datos a través de múltiples partidas, calcula el promedio de intentos sin incurrir en división por cero, y actualiza la mejor marca solo cuando una nueva partida supera el registro previo.

5.1.1 Verificación Experimental: Trazo de Ejecución para el Número 37

A continuación se presenta la traza completa de ejecución del algoritmo de búsqueda binaria para el valor secreto 37 —uno de los casos arbitrarios verificados—. La tabla reproduce fielmente el contenido del diccionario historial generado por el programa durante una partida real, incluyendo el snapshot del rango capturado como tupla en cada iteración antes de que los límites sean modificados:

Intento	Rango activo (snapshot tupla)	Conjetura — (bajo+alto)//2	Respuesta del usuario
1	[1, 100]	$50 \leftarrow (1+100)//2$	“menor” → alto = 49. Descartados 50 valores.

2	[1, 49]	$25 \leftarrow (1+49)/2$	“mayor” → bajo = 26. Descartados 25 valores.
3	[26, 49]	$37 \leftarrow (26+49)/2$	“correcto” → adivinado = True. Victoria en 3 intentos.

La traza confirma que el historial almacena en cada fila un diccionario con exactamente cuatro claves: ‘intento’ (int), ‘conjetura’ (int), ‘rango’ (tupla capturada antes de la actualización) y ‘respuesta’ (str). El número 37 fue identificado en solo 3 iteraciones —frente a los 7 garantizados como máximo—, lo que demuestra que el algoritmo escala de forma óptima: cuanto más cercano está el número al punto medio del rango activo, antes termina la búsqueda. Esta propiedad extrapolada a sistemas reales explica por qué los motores de búsqueda basados en índices ordenados responden en tiempo logarítmico independientemente del volumen de datos: el mismo principio que resuelve «Adivina el Número» en 3 pasos gobierna la búsqueda en bases de datos con millones de registros.

5.2 Resultados sobre el Uso de Estructuras de Datos

El análisis comparativo entre la versión anterior del programa —sin estructuras de datos— y la versión final revela los beneficios concretos aportados por cada estructura:

Estructura	Beneficio medible en el código
TUPLA RANGO	Eliminó la duplicación de los límites 1 y 100 en múltiples lugares del código. Antes, el mensaje de bienvenida y la inicialización del algoritmo eran independientes y podrían desincronizarse. Ahora, ambos derivan del mismo origen único.
TUPLA RESPUESTAS_VALIDAS	Centralizó la definición de las opciones válidas del juego. El mensaje de error ahora se genera automáticamente desde la tupla, garantizando coherencia. Anteriormente, las opciones se repetían tanto en la validación como en los mensajes de error.
LISTA historial	Habilitó la funcionalidad de presentación del trayecto completo de la partida, imposible de implementar sin una estructura de almacenamiento cronológico. El for loop sobre historial demuestra la utilidad del orden preservado de las listas.
DICCIONARIO registro	Estructuró los datos de cada intento con nombres semánticos ('conjetura', 'rango', 'respuesta') en lugar de posiciones numéricas anónimas. Esto hace el código de presentación del trayecto más legible y menos propenso a errores de indexación.
DICCIONARIO estadísticas	Reemplazó cuatro variables globales separadas (jugadas, ganadas, total, marca) por un único objeto con nombre, reduciendo la cantidad de variables en el espacio de nombres global y agrupando datos relacionados de forma cohesiva.
DICCIONARIO de retorno	Eliminó la necesidad de pasar múltiples valores de retorno mediante tuplas de posición —(True, 3, 72, [...])— que requieren recordar el orden. Ahora los valores se acceden por nombre: resultado['adivinado'], resultado['intentos'], resultado['historial'].

5.3 Resultado Académico

El proyecto demuestra el dominio integrado de todos los contenidos del semestre, sintetizados en un producto cohesivo y funcionalmente completo. La justificación semántica del uso de tuplas vs. listas evidencia comprensión profunda del lenguaje de programación, no solo su conocimiento superficial. La arquitectura de tres capas, el diseño modular y la documentación

técnica elevaron la calidad del código a estándares que trascienden el nivel universitario introductorio y se aproximan a prácticas de desarrollo profesional.

El análisis comparativo entre búsqueda lineal $O(n)$ y búsqueda binaria $O(\log_2 n)$ produjo una reflexión cuantitativa sobre la eficiencia algorítmica. Para un rango de 100 números: la búsqueda lineal requiere en promedio 50 intentos y 100 en el peor caso; la búsqueda binaria requiere un promedio de 5–6 intentos y un máximo de 7. Esta diferencia de un orden de magnitud introduce al estudiante en el pensamiento sobre complejidad computacional de forma intuitiva y verificable experimentalmente.

6. IMPLICACIONES

El desarrollo de este proyecto genera implicaciones que trascienden el ámbito de la tarea académica y se proyectan hacia el aprendizaje profesional, el diseño de software y la enseñanza de la informática.

6.1 Implicaciones Pedagógicas

La principal implicación pedagógica del proyecto es haber logrado una integración funcional —no cosmética— de las estructuras de datos. Esto tiene consecuencias importantes para el aprendizaje: cuando un estudiante utiliza una tupla porque comprendió que los límites del juego son inmutables, está ejerciendo pensamiento crítico sobre las propiedades del dato y eligiendo la herramienta correcta con base en criterios técnicos. Este nivel de razonamiento es cualitativamente diferente de «agregar una lista porque el enunciado lo pide».

El proyecto también implica que la enseñanza de estructuras de datos resulta más efectiva cuando se contextualiza dentro de un problema real. El estudiante que ve cómo `historial[-1]['conjetura']` recupera el número identificado al final de la partida comprende intuitivamente por qué la combinación de lista y diccionario es poderosa; esta comprensión es más duradera que la que se obtiene de ejercicios abstractos desconectados de un problema concreto.

6.2 Implicaciones para el Diseño de Software

El proyecto ilustra un principio fundamental del diseño de software conocido como «código que comunica intención»: el tipo de dato elegido debe revelar la naturaleza del dato. Cuando un programador lee `RESPUESTAS_VALIDAS: tuple = ('mayor', 'menor', 'correcto')`, el tipo `tuple` comunica de inmediato que estas opciones son fijas, que no se añadirán ni eliminarán durante la ejecución. Cuando lee `historial: list = []`, la lista vacía comunica que este será un registro que crece. Esta filosofía, aplicada consistentemente, reduce la carga cognitiva del mantenimiento de código y disminuye la probabilidad de errores.

El patrón de retorno de diccionario desde `jugar()` tiene implicaciones directas en el diseño de APIs. En el mundo profesional, las funciones que retornan múltiples valores relacionados frecuentemente lo hacen mediante diccionarios, objetos o estructuras de datos nombradas, precisamente porque el acceso por nombre es más robusto que el acceso por posición cuando el número de valores puede crecer en el futuro.

6.3 Implicaciones para la Carrera Profesional

El dominio demostrado en este proyecto —búsqueda binaria, estructuras de datos nativas, modularidad, documentación técnica y análisis de complejidad— corresponde a competencias evaluadas en entrevistas técnicas de empresas de tecnología. Algoritmos como la búsqueda binaria y estructuras de datos como listas y diccionarios forman parte del currículo estándar de plataformas de preparación técnica como LeetCode, HackerRank y Codility. El estudiante que domina estos conceptos desde el nivel introductorio construye una base sólida para cursos posteriores de estructuras de datos y algoritmos, y para el mercado laboral.

7. LIMITACIONES

El análisis honesto del sistema desarrollado requiere identificar sus limitaciones actuales con precisión y evidencia, distinguiendo entre limitaciones inherentes al diseño y las que pueden superarse en versiones futuras.

7.1 Limitaciones Funcionales

- Ausencia de verificación de la honestidad del usuario: El programa confía completamente en que el usuario responde las pistas correctamente. Si el usuario cambia de número durante la partida o responde incorrectamente, el algoritmo de búsqueda binaria puede agotarse sin encontrar el número. No existe ningún mecanismo de detección de inconsistencias en las respuestas del usuario. Una versión futura podría implementar validación de consistencia lógica: si el usuario responde «mayor» cuando el rango activo ya no tiene elementos superiores a la conjetura, el sistema podría alertar sobre una posible contradicción.
- Interfaz exclusivamente de consola: El programa opera únicamente mediante entrada/salida de texto en la terminal, lo que limita su accesibilidad para usuarios sin familiaridad con entornos de línea de comandos. No existe interfaz gráfica, visualización del rango como barra de progreso, ni posibilidad de deshacer una respuesta incorrecta. Esto reduce el atractivo del juego para audiencias no técnicas y limita su uso en contextos educativos con estudiantes de primaria o secundaria.
- Volatilidad de los datos: El historial de partidas y las estadísticas de sesión existen exclusivamente en memoria RAM durante la ejecución del programa. Al cerrarlo, toda la información se pierde. No existe mecanismo de persistencia que permita comparar resultados entre sesiones distintas o construir un tablero de posiciones histórico.
- Ausencia de manejo formal de excepciones: El programa no utiliza bloques try/except para gestionar errores inesperados, como interrupciones del teclado (Ctrl+C) o redirección de entrada desde archivo. Una interrupción en mitad de una partida dejará el programa en un estado inconsistente sin guardar los datos de la sesión.

7.2 Limitaciones Técnicas

- Rango fijo de 1 a 100: Aunque RANGO es una constante fácilmente modificable, el programa actual no ofrece al usuario la posibilidad de elegir el rango al inicio de cada

partida. Un rango configurable por el usuario requeriría validación adicional para asegurar que el límite inferior sea menor que el superior.

- Sin soporte de concurrencia: El programa es secuencial y síncrono; no puede atender múltiples usuarios simultáneos ni ejecutar varias partidas en paralelo. En un entorno educativo donde varios estudiantes quisieran usar el juego simultáneamente, se requeriría una arquitectura cliente-servidor con manejo de estado por sesión.
- Sin pruebas unitarias automatizadas: El programa carece de un conjunto de pruebas unitarias que verifiquen automáticamente el comportamiento correcto del algoritmo para todos los valores posibles. La verificación actual es manual, lo que deja espacio para la introducción accidental de regresiones si el código es modificado.

7.3 Limitaciones de Alcance Académico

El proyecto se enmarca en un curso introductorio de lógica de programación, por lo que no aborda paradigmas de programación orientada a objetos —en cuyo caso la partida podría encapsularse en una clase `Partida` con atributos y métodos—, ni técnicas de programación funcional que harían el código más conciso con funciones como `filter()` o `map()`. Estas limitaciones son apropiadas para el nivel del curso pero deben ser reconocidas como oportunidades de profundización en cursos subsiguientes.

8. REFLEXIÓN SOBRE EL IMPACTO TECNOLÓGICO EN LA SOCIEDAD

El algoritmo de búsqueda binaria y el pensamiento computacional que lo sustenta no son abstracciones académicas confinadas a los libros de texto; son principios que gobiernan sistemas tecnológicos que millones de personas utilizan cotidianamente sin ser conscientes de su existencia.

Cuando un usuario busca un contacto en su teléfono, un producto en un catálogo digital o una palabra en un diccionario electrónico, los algoritmos de búsqueda subyacentes —muchos de ellos variantes o generalizaciones de la búsqueda binaria— procesan millones de registros en fracciones de segundo. La diferencia entre un algoritmo $O(n)$ y un algoritmo $O(\log n)$ no es teórica: para una base de datos con un millón de registros, la búsqueda lineal podría necesitar un millón de comparaciones, mientras que la búsqueda binaria necesita apenas veinte. Esta diferencia determina si una aplicación responde en tiempo real o hace esperar al usuario varios segundos.

Más allá del rendimiento técnico, el proyecto invita a reflexionar sobre la democratización del conocimiento algorítmico. Históricamente, el diseño de algoritmos eficientes era dominio exclusivo de matemáticos y científicos computacionales con formación altamente especializada. La disponibilidad de lenguajes de alto nivel como Python, combinada con recursos educativos accesibles, ha democratizado ese conocimiento: un estudiante de primer año puede hoy implementar y comprender un algoritmo de complejidad logarítmica que hace décadas habría requerido conocimiento profundo de matemáticas discretas.

Esta democratización tiene implicaciones sociales profundas. Los países que logren formar masas críticas de ciudadanos con pensamiento algorítmico —no necesariamente programadores profesionales— tendrán ventajas significativas en el diseño de políticas públicas basadas en datos, en la automatización de procesos industriales y en la participación informada en debates sobre inteligencia artificial, privacidad digital y ética tecnológica.

Sin embargo, esta democratización conlleva responsabilidades. La facilidad con que Python permite implementar sistemas complejos puede llevar a programadores inexpertos a desplegar soluciones en producción sin comprender sus limitaciones de escala, seguridad o equidad algorítmica. Un algoritmo que no trate los datos de todos los usuarios de forma equitativa puede perpetuar o amplificar sesgos existentes. La formación en lógica de programación debe incluir, por tanto, no solo el dominio técnico sino también el pensamiento crítico sobre las consecuencias sociales de las decisiones de diseño.

El juego «Adivina el Número», en su aparente simplicidad, ilustra un principio que trasciende la programación: la estrategia óptima para resolver un problema no siempre es la más obvia. Buscar el número secreto probando valores uno a uno es la estrategia intuitiva; dividir el espacio de búsqueda sistemáticamente es la estrategia inteligente. Esta metáfora se aplica a innumerables problemas de la vida real: en medicina, el diagnóstico diferencial sigue principios similares; en ingeniería, la búsqueda de la causa raíz de un fallo; en economía, la fijación óptima de precios. El pensamiento algorítmico es, en última instancia, una forma de pensar más eficientemente sobre cualquier problema que requiera búsqueda o decisión bajo incertidumbre.

9. CONCLUSIONES

El desarrollo del proyecto «Adivina el Número» en su versión final con estructuras de datos integradas conduce a las siguientes conclusiones, organizadas desde la perspectiva técnica, pedagógica y de ingeniería de software:

1. La búsqueda binaria es una estrategia óptima garantizada para el problema de adivinanza: la complejidad $O(\log_2 n)$ asegura matemáticamente que ningún algoritmo comparativo puede resolver el problema en menos de $\lceil \log_2 n \rceil$ comparaciones en el peor caso. Para $n = 100$, esto se traduce en un máximo de 7 intentos, verificado experimentalmente en múltiples pruebas con valores extremos, centrales y aleatorios.

2. Las estructuras de datos no son intercambiables: la elección entre tupla y lista, o entre lista y diccionario, debe responder a las propiedades del dato que representan. Las tuplas son apropiadas para constantes del dominio que no deben modificarse; las listas, para colecciones que crecen dinámicamente; y los diccionarios, para datos con nombre cuyo acceso semántico por clave es más significativo que el acceso posicional.

3. La modularidad reduce la complejidad cognitiva del sistema: encapsular la lógica de una partida en `jugar()` y comunicar resultados mediante un diccionario de retorno elimina la necesidad de variables globales, hace el código más fácil de entender, de probar y de reutilizar.

La ausencia de efectos secundarios globales es una propiedad valiosa que debe cultivarse desde los primeros niveles de formación en programación.

4. La arquitectura en tres capas, incluso en un programa de consola, aporta claridad: separar presentación, lógica de negocio y datos —aunque en un mismo archivo— permite comprender cuáles partes del sistema son candidatas a evolucionar independientemente. Esta separación es el germen de arquitecturas más complejas que el estudiante encontrará en cursos de ingeniería de software, desarrollo web y sistemas distribuidos.

5. La documentación técnica es parte inseparable de la calidad del código: el módulo `docstring` con la tabla de estructuras de datos, las anotaciones de tipo y los comentarios que explican el «por qué» de las decisiones de diseño —no el «qué» obvio del código— convierten el programa en un artefacto comprensible para cualquier programador, independientemente de si participó en su desarrollo.

6. El pensamiento algorítmico tiene proyección directa en el mundo real: los principios demostrados en este proyecto —eficiencia logarítmica, diseño modular, estructuras de datos apropiadas— son los mismos que gobiernan sistemas de búsqueda, clasificación y procesamiento de datos que sustentan aplicaciones usadas por millones de personas. Dominar estos principios desde el nivel introductorio sienta las bases para contribuir al desarrollo tecnológico con criterio y responsabilidad.

7. Las limitaciones identificadas señalan el camino hacia versiones mejoradas: la ausencia de persistencia de datos, la interfaz de consola, la falta de verificación de consistencia del usuario y la ausencia de pruebas automatizadas no son deficiencias del diseño actual sino oportunidades de aprendizaje claramente delimitadas, cada una asociada a competencias específicas que el estudiante desarrollará en semestres posteriores.

10. RECOMENDACIONES

Con base en el análisis, los resultados y las limitaciones identificadas, se formulan las siguientes recomendaciones orientadas a mejorar el sistema y a guiar el aprendizaje futuro:

10.1 Recomendaciones Técnicas para el Sistema

- Implementar persistencia de datos mediante el módulo `json` de Python: escribir el diccionario estadísticas en un archivo JSON al finalizar cada sesión y cargarlo al inicio de la siguiente permitiría construir un historial de partidas persistente entre sesiones y calcular estadísticas de largo plazo.
- Agregar manejo de excepciones con bloques `try/except` para capturar `KeyboardInterrupt` y `EOFError`, guardando el estado de la sesión antes de cerrar el programa cuando el usuario interrumpe la ejecución con `Ctrl+C`. Esto mejoraría la robustez del sistema en condiciones de uso real.
- Desarrollar un módulo de pruebas unitarias utilizando `pytest` o el módulo `unittest` de la biblioteca estándar de Python. Las pruebas deberían verificar automáticamente que

jugar() identifica correctamente cada número del 1 al 100 en un máximo de 7 intentos, simulando las respuestas del usuario mediante mocking.

- Considerar la adición de un modo donde la computadora elige el número secreto aleatoriamente —usando el módulo random— y el usuario intenta adivinarlo. Este modo inverso reutilizaría las mismas estructuras de datos y el bloque de estadísticas, con modificaciones mínimas en la lógica de retroalimentación.

10.2 Recomendaciones Pedagógicas

- Profundizar en programación orientada a objetos refactorizando el programa: crear una clase Partida que encapsule el estado de la búsqueda binaria, y una clase Sesion que gestione múltiples partidas y estadísticas. Esta refactorización ilustrará cómo las mismas estructuras de datos se integran naturalmente en objetos.
- Explorar las diferencias de rendimiento experimentalmente: medir el tiempo de ejecución de búsqueda binaria vs. búsqueda lineal para rangos de 100, 1.000, 10.000 y 1.000.000 usando el módulo timeit, y graficar los resultados. Esta experiencia convertiría la complejidad algorítmica de concepto teórico en dato empírico verificable.
- Diseñar una interfaz gráfica con Tkinter como proyecto de semestre siguiente, reutilizando la función jugar() sin modificación como back-end. Este ejercicio demostrará el valor de la separación entre lógica y presentación, y motivará la adopción de arquitecturas limpias desde las etapas más tempranas del aprendizaje.
- Estudiar las colecciones especializadas del módulo collections de Python —Counter, defaultdict, OrderedDict, deque— como extensión natural de los diccionarios y listas básicos aprendidos en este proyecto, para ampliar el repertorio de estructuras de datos disponibles según el tipo de problema.

10.3 Recomendación Final

La recomendación más importante que emerge de este proyecto es de orden actitudinal: cultivar el hábito de justificar cada decisión de diseño. Cuando se elige una tupla en lugar de una lista, cuando se diseña una función con valor de retorno en lugar de modificar una variable global, cuando se documenta el «por qué» de una decisión y no solo el «qué», se está practicando el pensamiento de ingeniería. Esta actitud reflexiva y crítica ante el código propio —evaluarlo no solo por si funciona, sino por si está bien diseñado— es la que diferencia a un programador competente de un ingeniero de software de excelencia.

REFERENCIAS BIBLIOGRÁFICAS

-
1. Python Software Foundation. (2026). Python 3.14 Documentation. <https://docs.python.org/3/>
 2. Python Software Foundation. (2026). Built-in Types — Python 3.14. <https://docs.python.org/3/library/stdtypes.html>
 3. Microsoft Corporation. (2026). Visual Studio Code v1.110 Release Notes. <https://code.visualstudio.com/updates/>

4. Carlisle, M. C., Wilson, T. A., Humphries, J. W., & Hadfield, S. M. (2004). RAPTOR: A Visual Programming Environment for Teaching Algorithmic Problem Solving. Proceedings of the 35th SIGCSE Technical Symposium on Computer Science Education.

ANEXO A — CÓDIGO FUENTE COMPLETO DEL PROGRAMA

A continuación, se presenta el código fuente completo del proyecto “Adivina el Número” en Python 3.14, desarrollado en Visual Studio Code 1.110. El programa implementa el algoritmo de búsqueda binaria integrando tuplas, listas y diccionarios con roles semánticos diferenciados, conforme a la descripción detallada en el cuerpo del documento.

““

ADIVINA EL NÚMERO — Juego de Búsqueda Binaria con Estructuras de Datos

Proyecto Integrador de Lógica de Programación — Semana 8

Universidad Internacional del Ecuador — UIDE

Estudiante: Mery Elizabeth Guzmán Ontaneda

Estructuras de datos:

tuple: RANGO, RESPUESTAS_VALIDAS, rango_snapshot (constantes e inmutables)

list : historial (registro cronológico de intentos)

dict : registro por intento, estadísticas de sesión, retorno de jugar()

— CONSTANTES DE MÓDULO

RANGO: tuple = (1, 100) # Límites inmutables del espacio de búsqueda

RESPUESTAS_VALIDAS: tuple = ('mayor', 'menor', 'correcto')

MAX_INTENTOS: int = 7 # $\lceil \log_2(100) \rceil = 7$ garantizados

— FUNCIÓN PRINCIPAL

def jugar() -> dict:

"""Ejecuta una partida. Retorna dict con: adivinado, intentos, numero, historial."""

Paso 1 – Inicialización

bajo, alto = RANGO # Desempaquetado de tupla

intentos: int = 0

adivinado: bool = False

historial: list = [] # Lista dinámica vacía

print(f"\n La computadora adivina TU número entre {RANGO[0]} y {RANGO[1]}")

print(f" Máximo garantizado: {MAX_INTENTOS} intentos")

Paso 2 – Bucle de búsqueda binaria

while not adivinado and intentos < MAX_INTENTOS:

intentos += 1

medio: int = (bajo + alto) // 2

rango_snapshot: tuple = (bajo, alto) # Captura inmutable del estado

print(f"\n Intento {intentos}/{MAX_INTENTOS} | Rango: [{bajo}–{alto}]")

print(f" ¿Tu número es ... {medio}?)")

Validación de entrada (bucle interno)

respuesta_valida: bool = False

```

while not respuesta_valida:
    respuesta = input(" Tu respuesta: ").strip().lower()
    if respuesta in RESPUESTAS_VALIDAS: # Operador 'in' sobre tupla
        respuesta_valida = True
    else:
        opts = ' | '.join(RESPUESTAS_VALIDAS) # join() dinámico sobre tupla
        print(f" Opciones válidas: {opts}")
# Clasificación de la respuesta y actualización del rango
if respuesta == 'correcto':
    adivinado = True
elif respuesta == 'mayor':
    bajo = medio + 1
else: # 'menor'
    alto = medio - 1
# Registro del intento como diccionario → se añade a la lista historial
registro: dict = {
    'intento' : intentos,
    'conjetura': medio,
    'rango'   : rango_snapshot, # Tupla anidada en diccionario
    'respuesta': respuesta
}
historial.append(registro) # Lista crece O(1) amortizado
# Presentación del trayecto (for loop sobre lista de diccionarios)
for reg in historial:
    inicio, fin = reg['rango'] # Desempaquetado de tupla anidada
    print(f" {reg['intento']}>3} [{inicio:>3}–{fin:<3}] {reg['conjetura']:>6}
{reg['respuesta']}")
# Retorno múltiple encapsulado en diccionario (evita variables globales)
return {
    'adivinado': adivinado,
    'intentos' : intentos,
    'numero'   : historial[-1]['conjetura'] if adivinado else None,
    'historial': historial
}
# — BLOQUE DE EJECUCIÓN

```

```

if __name__ == '__main__':
    estadisticas: dict = {
        'partidas_jugadas': 0, 'partidas_ganadas': 0,
        'total_intentos': 0, 'mejor_marca': MAX_INTENTOS + 1
    }
    continuar: bool = True
    while continuar:

```

```

resultado = jugar()
estadisticas['partidas_jugadas'] += 1
estadisticas['total_intentos'] += resultado['intentos']
if resultado['adivinado']:
    estadisticas['partidas_ganadas'] += 1
    if resultado['intentos'] < estadisticas['mejor_marca']:
        estadisticas['mejor_marca'] = resultado['intentos']
respuesta = input("\n ¿Deseas jugar de nuevo? (s/n): ").strip().lower()
continuar = (respuesta == 's')
# Resumen final de sesión
jugadas = estadisticas['partidas_jugadas']
if jugadas > 0:
    # Evita división por cero
    promedio = estadisticas['total_intentos'] / jugadas
    print(f" Partidas jugadas : {estadisticas['partidas_jugadas']}")
    print(f" Partidas ganadas : {estadisticas['partidas_ganadas']}")
    print(f" Promedio intentos: {promedio:.1f}")
    if estadisticas['mejor_marca'] <= MAX_INTENTOS:
        print(f" Mejor marca      : {estadisticas['mejor_marca']} intento(s)")
print("\n ¡Hasta pronto!\n")

```